CTE711S Compiler Techniques

Week 3

Syntax Analysis and Context-Free Grammar (CFG)

Prof A. A. Azeta (aazeta@nust.na)

February 2026

Recommended Texts for this Course

1. Watson, Des (2017). **A Practical Approach to Compiler Construction**, Springer. [MainText]

2. Campbell, Bill; Iyer, Swami; and Akbal-Delibas, Bahar (2013). **Introduction to Compiler Construction in a Java World**, Taylor & Francis Group.

3. Seidl, Helmut; Wilhelm, Reinhard; and Hack, Sebastian (2012). **Compiler Design: Analysis and Transformation**, Springer

4. Grune, Dick; Reeuwijk, Kees van; Bal, Henri E; Jacobs, Ceriel J.H; and Langendoen, Koen (2012). **Modern Compiler Design**, Second Edition, Springer.

5. Reis, Anthony Dos (2011). **Compiler Construction Using Java, JavaCC, and Yacc**, Wiley

Recap: From Description to Implementation

- ✓ **Lexical analysis (Scanning):** Identify logical pieces of the description.
- ✓ **Syntax analysis (Parsing):** Identify how those pieces relate to each other.
- ✓ **Semantic analysis:** Identify the meaning of the overall structure.
- ✓ **IR Generation:** Design one possible structure.
- ✓ **IR Optimization:** Simplify the intended structure.
- ✓ **Generation:** Fabricate the structure.
- ✓ **Optimization:** Improve the resulting structure.

Tokens Versus Terminals

- In a compiler, the lexical analyzer reads the characters of the source program, groups them into lexically meaningful units called lexemes, and produces as output tokens representing these lexemes.
- A token consists of two components, a token name and an attribute value.
- The token names are abstract symbols that are used by the parser for syntax analysis.
- Often, we shall call these token names ***terminals***, since they appear as terminal symbols in the grammar for a programming language.
- The attribute value, if present, is a *pointer* to the symbol table that contains additional information about the token.

In-Class Exercise 1

```
int main ()
{
    int a = 0;
    cout << a << endl;
    return 1;
}
```

- Given the sample code above, list and describe the tokens.

Tokens for In-Class Exercise 1

- Identifier (int)
- Identifier (main)
- Left parenthesis
- Right parenthesis
- Left curly brace
- Identifier (int)
- Equals
- Integer (0)
- Semi-colon
- Identifier (cout)
- Double left angle bracket
- Identifier (a)
- Double left angle bracket
- Identifier (endl)
- Semi-colon
- Reserved word (return)
- Integer (1)
- Semi-colon
- Right curly brace

Tree Terminology

❖ Tree data structures figure prominently in compiling.

- A tree consists of one or more *nodes.* Nodes may have *labels,* which typically will be grammar symbols.When we draw a tree, we often represent the nodes by these labels only.
- Exactly one node is the *root.* All nodes except the root have a unique *parent;* the root has no parent. When we draw trees, we place the parent of a node above that node and draw an edge between them. The root is then the highest (top) node.
- If node *N* is the parent of node *M,* then *M* is a *child* of *N.* The children of one node are called *siblings.* They have an order, *from the left,* and when we draw trees, we order the children of a given node in this manner.
- A node with no children is called a *leaf.* Other nodes — those with one or more children — are *interior nodes.*
- A *descendant* of a node *N* is either *N* itself, a child of *N,* a child of a child of *N,* and so on, for any number of levels.We say node *N* is an *ancestor* of node *M* if *M* is a descendant of *N.*

Definition of Grammars

A *context-free grammar (CFG)* has four components

1.A set of ***terminal*** symbols, sometimes referred to as "tokens." The terminals are the elementary symbols of the language defined by the grammar.

2.A set of ***nonterminals***, sometimes called "syntactic variables." Each nonterminal represents a set of strings of terminals, in a manner we shall describe.

3.A set of ***productions***, where each production consists of a nonterminal, called the *head* or *left side* of the production, an arrow, and a sequence of terminals and/or nonterminals, called the *body* or *right side* of the production. The intuitive intent of a production is to specify one of the written forms of a construct; if the head nonterminal represents a construct, then the body represents a written form of the construct.

4.A designation of one of the nonterminals as the *START* symbol.

Syntax Definition

- Context-free Grammar is a 4-tuple with
  - A set of tokens (*terminal* symbols), **T**
  - A set of *nonterminals*, ***N***
  - A set of *productions*, ***P***

$X \rightarrow Y_1 Y_2 \ldots Y_n$

where $X \in N$ and $Y_i \in T \cup N \cup \{\varepsilon\}$

  - A designated *start symbol* ***S*** *(a non-terminal)*

Example CFGs: Simple Arithmetic Expressions

In English:

- An integer is an arithmetic expression.
- If exp_1 and exp_2 are arithmetic expressions, then so are the following:
  - exp_1 - exp_2
  - exp_1 / exp_2
  - (exp_1)

the corresponding CFG: we'll write tokens as follows:

- exp → INTLITERAL E → intlit
- exp → exp MINUS exp E → E - E
- exp → exp DIVIDE exp E → E / E
- exp → LPAREN exp RPAREN E → (E)

Example of a Grammar

- Context-free grammar for simple expressions:

G = <{*list*,*digit*}, {**+**,**-**,**0**,**1**,**2**,**3**,**4**,**5**,**6**,**7**,**8**,**9**}, *P*, *list* >

with productions *P* =

list → *list* **+** *digit*
list → *list* **-** *digit*
list → *digit*
digit → **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9**

Derivation

- Given a CF grammar we can determine the set of all *strings* (sequences of tokens) generated by the grammar using *derivation*

- A derivation is a sequence of productions, P → … → … → …
  - A derivation can be drawn as a Tree
  - Start symbol is the tree's root X
  - For a production X → Y1 … Yn add children Y1 … Yn to node

- We begin with the start symbol, *S*
- In each step, we replace one nonterminal in the current *sentential form* with one of the right-hand sides of a production for that nonterminal

Example of a Grammar

- Context-free grammar for simple expressions:

G = <{*list*, *digit*}, {**+**,**-**,**0**,**1**,**2**,**3**,**4**,**5**,**6**,**7**,**8**,**9**}, *P*, *list* >

with productions *P* =

list → *list* + *digit*

list → *list* **-** *digit*

list → *digit*

digit → **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9**

Derivation for the Example Grammar

$\underline{list}$

$\Rightarrow \underline{list} + digit$

$\Rightarrow \underline{list} - digit + digit$

$\Rightarrow \underline{digit} - digit + digit$

$\Rightarrow 9 - \underline{digit} + digit$

$\Rightarrow 9 - 5 + \underline{digit}$

$\Rightarrow 9 - 5 + 2$

This is an example *leftmost derivation*, because we replaced the leftmost nonterminal (underlined) in each step.
Likewise, a *rightmost derivation* replaces the rightmost nonterminal in each step

Derivation: An example

❖ CFG:

E → id

E → E + E

E → E * E

E → (E)

❖ derivation:

E→ E+E

→ E*E+E

→ id*E+E

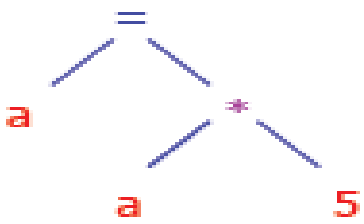
→ id*id+E

→ id*id+id

- Is string id * id + id in the language defined by grammar?

Syntax Analyzer (Parser)

- Groups tokens together to form grammatical phrases
  - Builds a structure to capture the program (Abstract Syntax Tree)
  - Interior nodes – operators
  - Children - operands
- Example: a = a * 5;

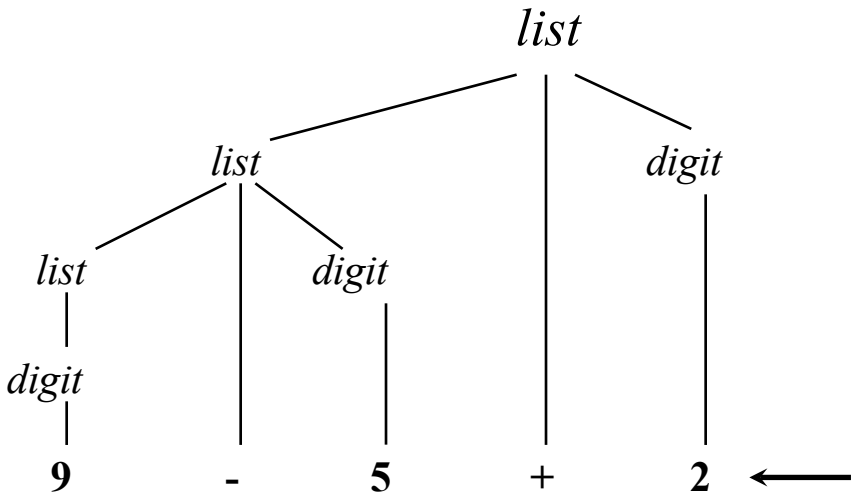


Parse Trees

- The ***root*** of the tree is labeled by the *Start* symbol
- Each ***leaf*** of the tree is labeled by a *Terminal* (=token) or ε
- Each ***interior node*** is labeled by a *Nonterminal*
- If $A \rightarrow X_1 X_2 \ldots X_n$ is a production, then node A has immediate *children* X_1, $X_2, \ldots, X_n$ where X_i is a (non)terminal or ε (ε denotes the *empty string*)

Parse Tree for the Example Grammar

Parse tree of the string **9-5+2** using grammar, ***G***



The sequence of leafs is called the *yield* of the parse tree

Ambiguity

- Consider the following context-free grammar:

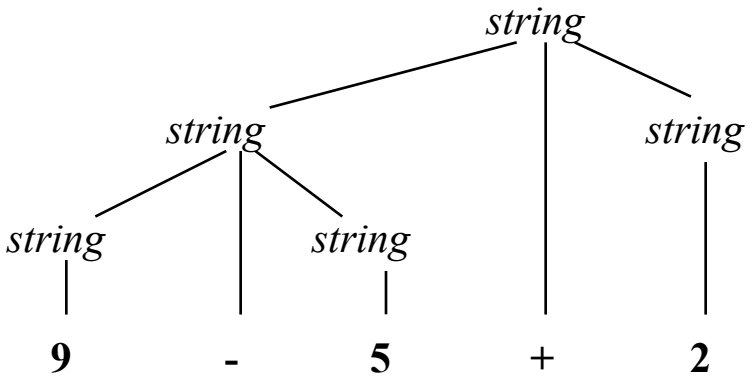
$G = <\{string\}, \{\mathbf{+,-,0,1,2,3,4,5,6,7,8,9}\}, P, string>$

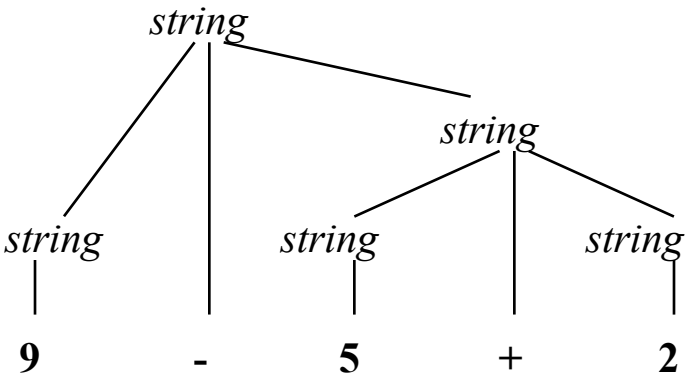
with production $P =$

$string \rightarrow string \mathbf{+} string \mid string \mathbf{-} string \mid \mathbf{0} \mid \mathbf{1} \mid \ldots \mid \mathbf{9}$

This grammar is *ambiguous*, because more than one parse tree represents the string **9-5+2**

Ambiguity (cont'd)





Associativity of Operators

- ***Left-associative*** operators have *left-recursive* productions

  left → *left* + *term* | *term*

  String **a+b+c** has the same meaning as **(a+b)+c**

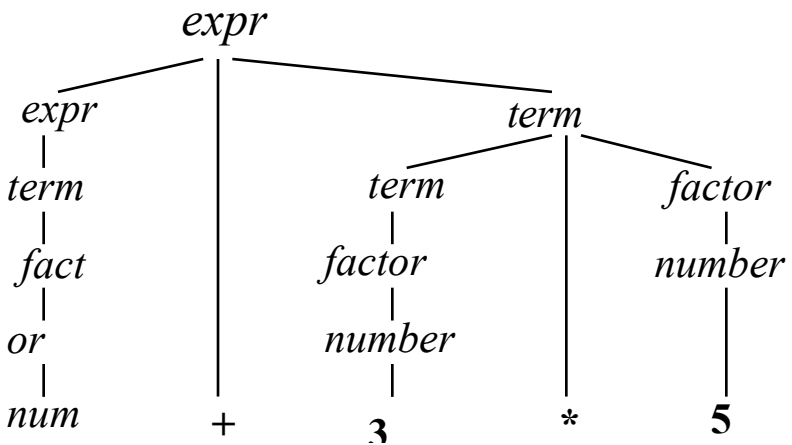
- ***Right-associative*** operators have *right-recursive* productions

  right → *term* = *right* | *term*

  String **a=b=c** has the same meaning as **a=(b=c)**

Precedence of Operators

- Operators with higher precedence "bind more tightly"
  - *expr* → *expr* + *term* | *term*
  - *term* →*term* * *factor* | *factor*
  - *factor* → *number* | **(** *expr* **)**
  - String **2+3*5** has the same meaning as **2+(3*5)**



The Parse Tree

Syntax of Statements

stmt → **id :=** *expr*

| **if** *expr* **then** *stmt*

| **if** *expr* **then** *stmt* **else** *stmt*

| **while** *expr* **do** *stmt*

| **begin** *opt_stmts* **end**

opt_stmts → *stmt* **;**

opt_stmts | ε

Syntax-Directed Translation (SDT)

- Uses a CFG to specify the syntactic structure of the language.
- AND associates a set of *attributes* with the terminals and nonterminals of the grammar
- AND associates with each production a set of *semantic rules* to compute values of attributes
- A parse tree is traversed and semantic rules applied:
  - After the computations are completed the attributes contain the translated form of the input

Synthesized and Inherited Attributes

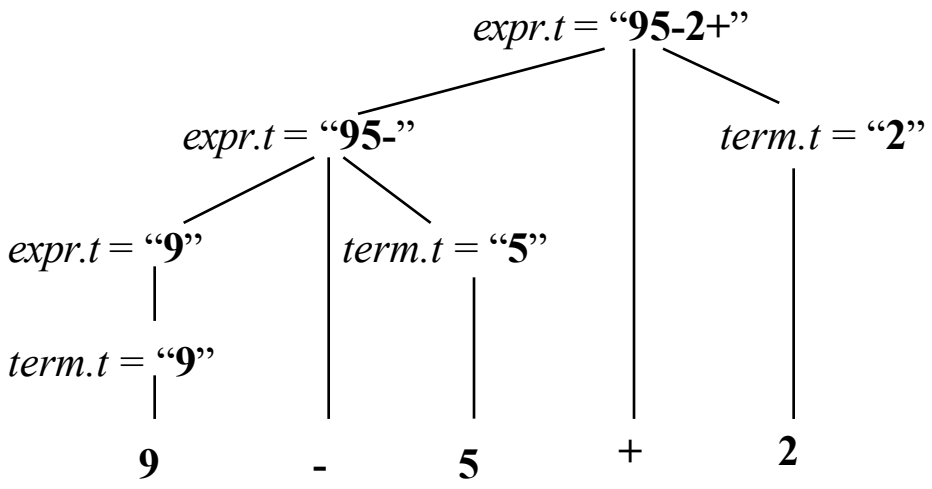
- An attribute is said to be:
  - ***synthesized*** if its value at a parse-tree node is determined from the attribute values at the children of the node
  - ***inherited*** if its value at a parse-tree node is determined by the parent (by enforcing the parent's semantic rules)

Example Attribute Grammar

String concat operator

Production	Semantic Rule
$expr \rightarrow expr_1 + term$	$expr.t := expr_1.t \mathbin{//} term.t \mathbin{//} \text{“+”}$
$expr \rightarrow expr_1 - term$	$expr.t := expr_1.t \mathbin{//} term.t \mathbin{//} \text{“-”}$
$expr \rightarrow term$	$expr.t := term.t$
$term \rightarrow \mathbf{0}$	$term.t := \text{“0”}$
$term \rightarrow \mathbf{1}$	$term.t := \text{“1”}$
…	…
$term \rightarrow \mathbf{9}$	$term.t := \text{“9”}$

Example: Annotated Parse Tree



In-Class Exercise

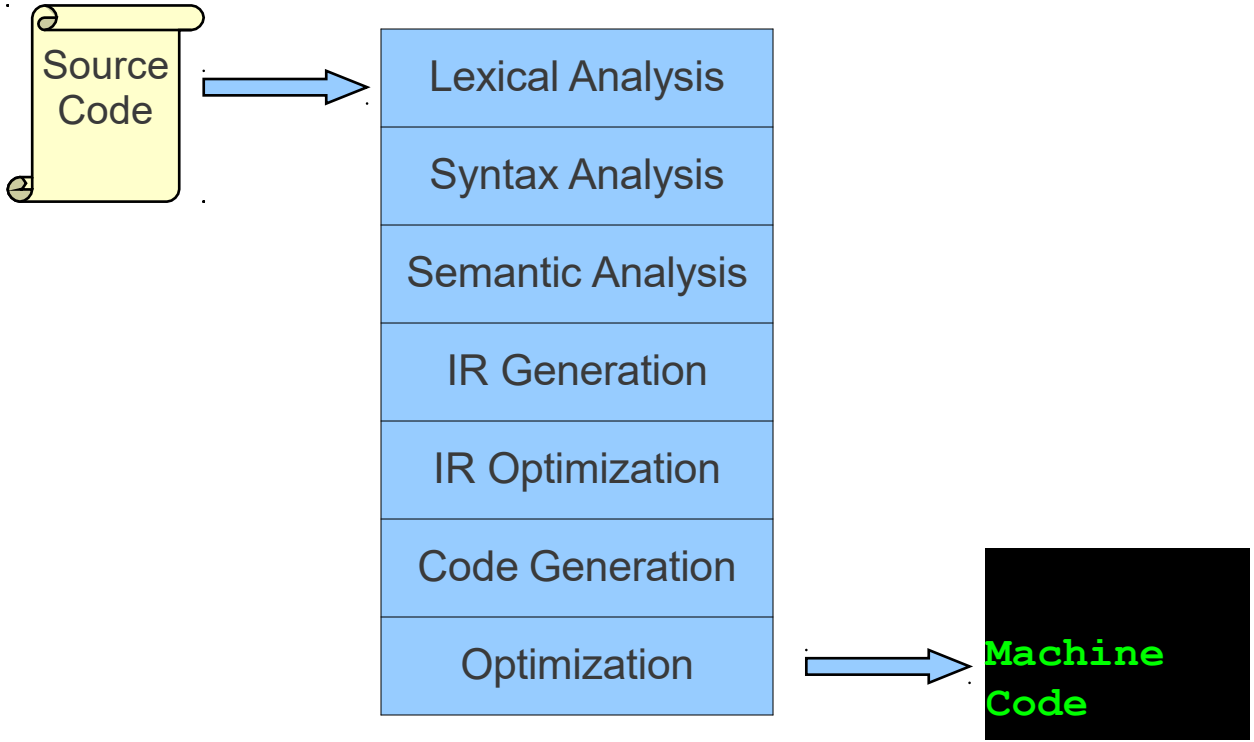
- Consider the Context-Free Grammar

$$G = \langle \{S\}, \{\mathbf{a}, -, *\}, P, S \rangle$$

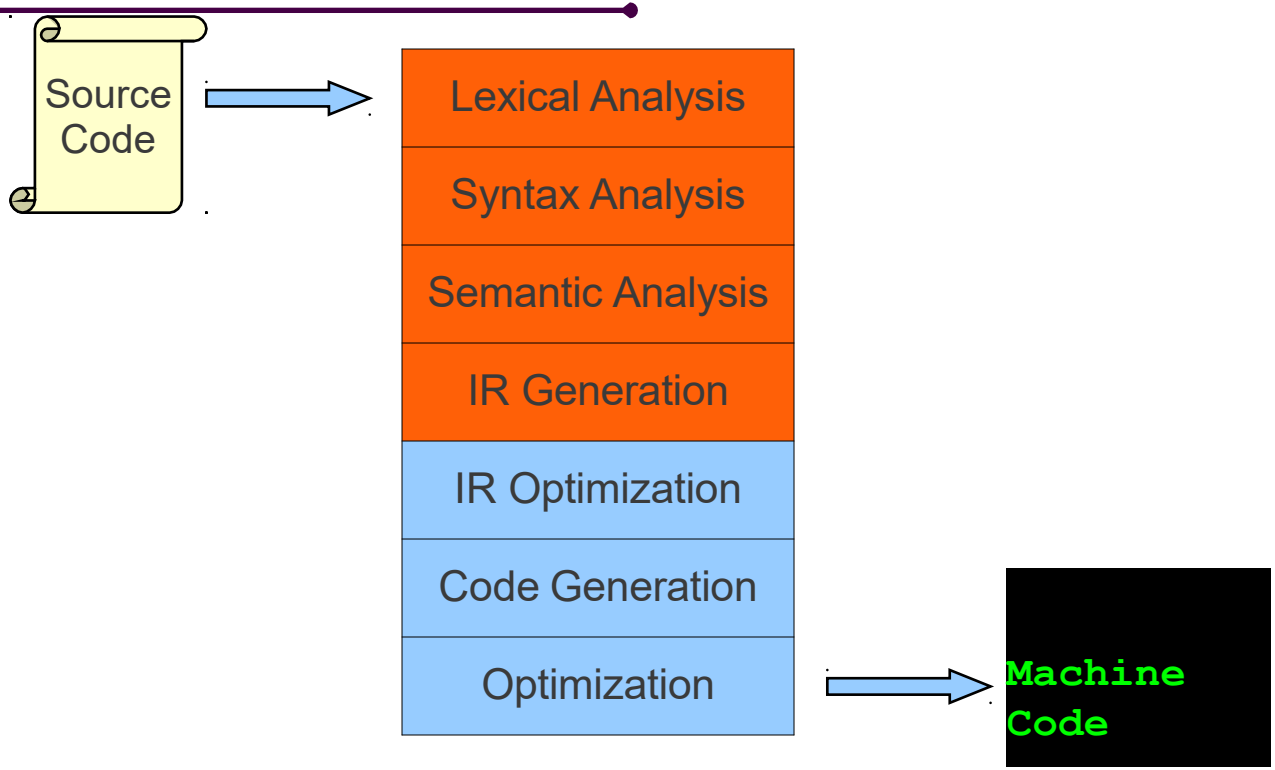
with productions P: $S \rightarrow S- \mid SS* \mid \mathbf{a}$

- Show how the string $\mathbf{a} - \mathbf{a} * - \mathbf{a} *$ can be generated by G (using derivation).

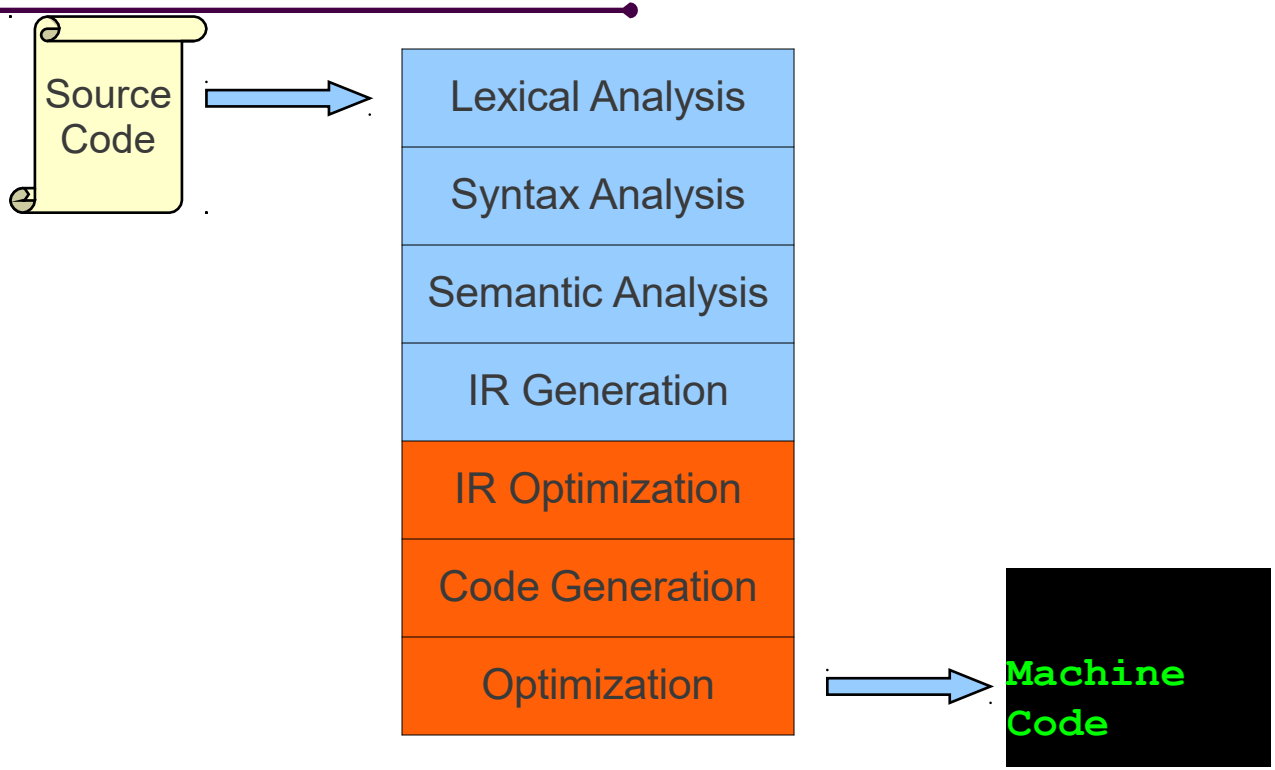
The Structure of a Typical Compiler



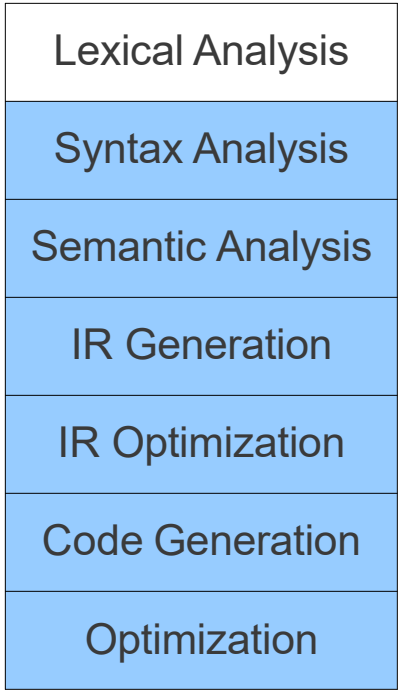
The Structure of a Modern Compiler



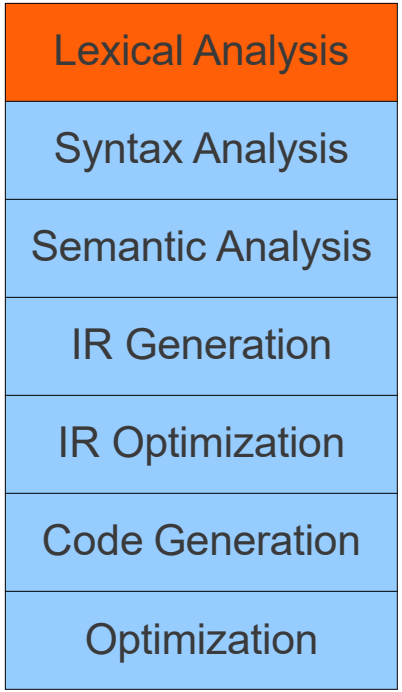
The Structure of a Modern Compiler



```
while (y < z)
{
  Int x  = a  + b;
  y+=x;
}
```

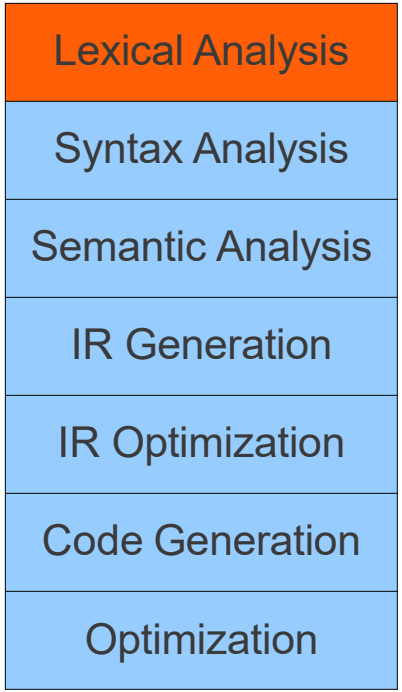


```
while (y < z)
{
int x = a + b;
y += x;
}
```



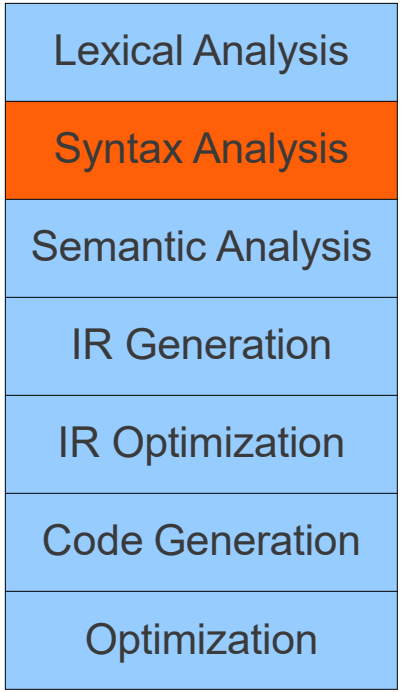
```
while (y < z)
{
  int x = a + b;
  y += x;
}
```

T_While
T_LeftParen
T_Identifier y
T_Less
T_Identifier z
T_RightParen
T_OpenBrace
T_Int
T_Identifier x
T_Assign
T_Identifier a
T_Plus
T_Identifier b
T_Semicolon
T_Identifier y
T_PlusAssign
T_Identifier x
T_Semicolon
T_CloseBrace

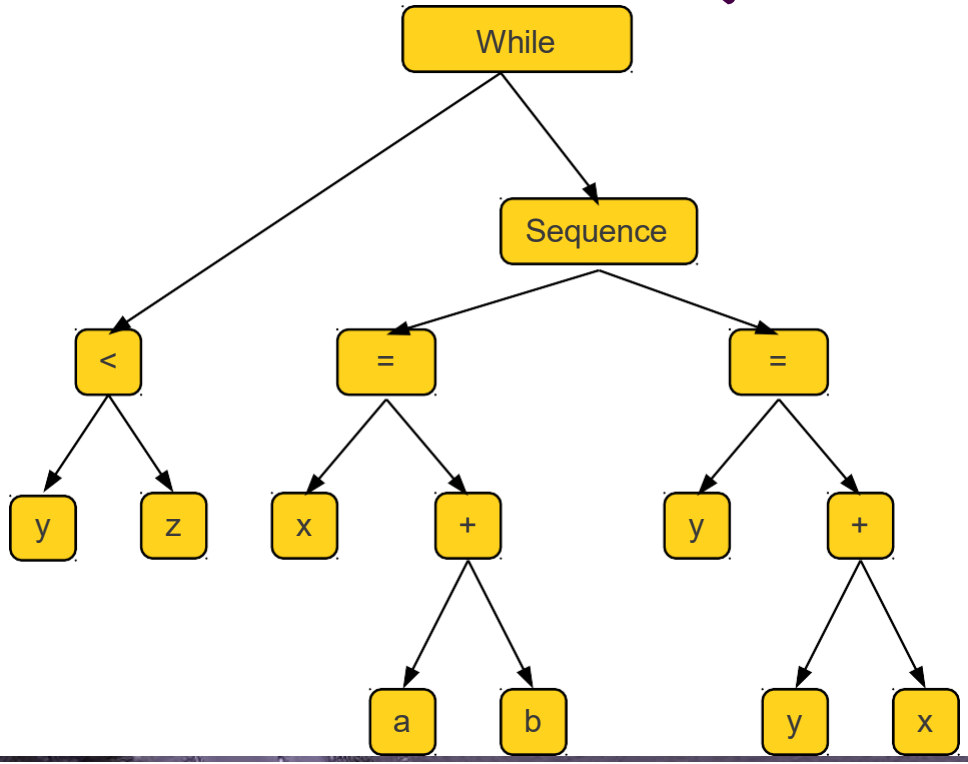


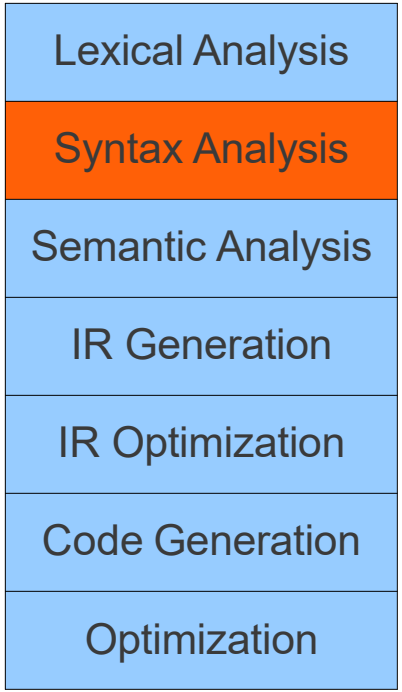
```
while (y < z)
{ int x = a + b;
  y += x;
}
```

```
T_While
T_LeftParen
T_Identifier y
T_Less
T_Identifier z
T_RightParen
T_OpenBrace
T_Int
T_Identifier x
T_Assign
T_Identifier a
T_Plus
T_Identifier b
T_Semicolon
T_Identifier y
T_PlusAssign
T_Identifier x
T_Semicolon
T_CloseBrace
```

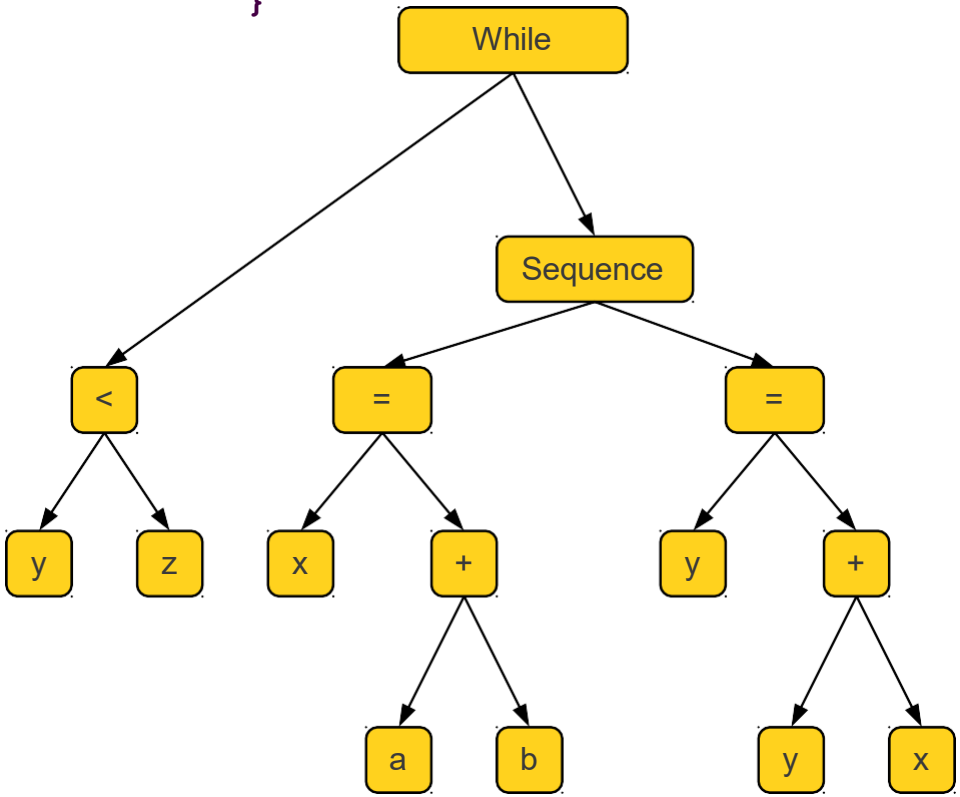


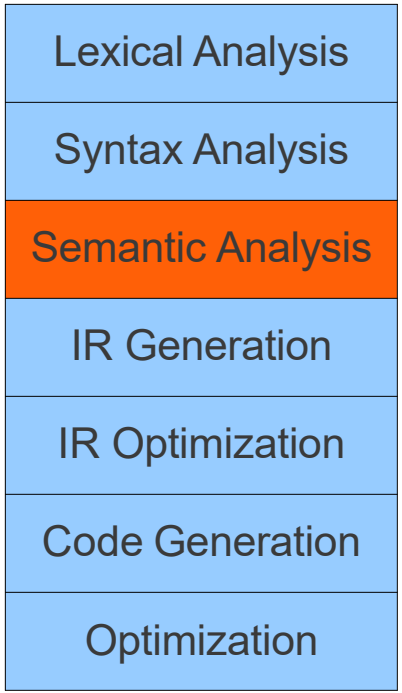
```
while (y < z) {
 int x = a + b;
     y += x;
       }
```



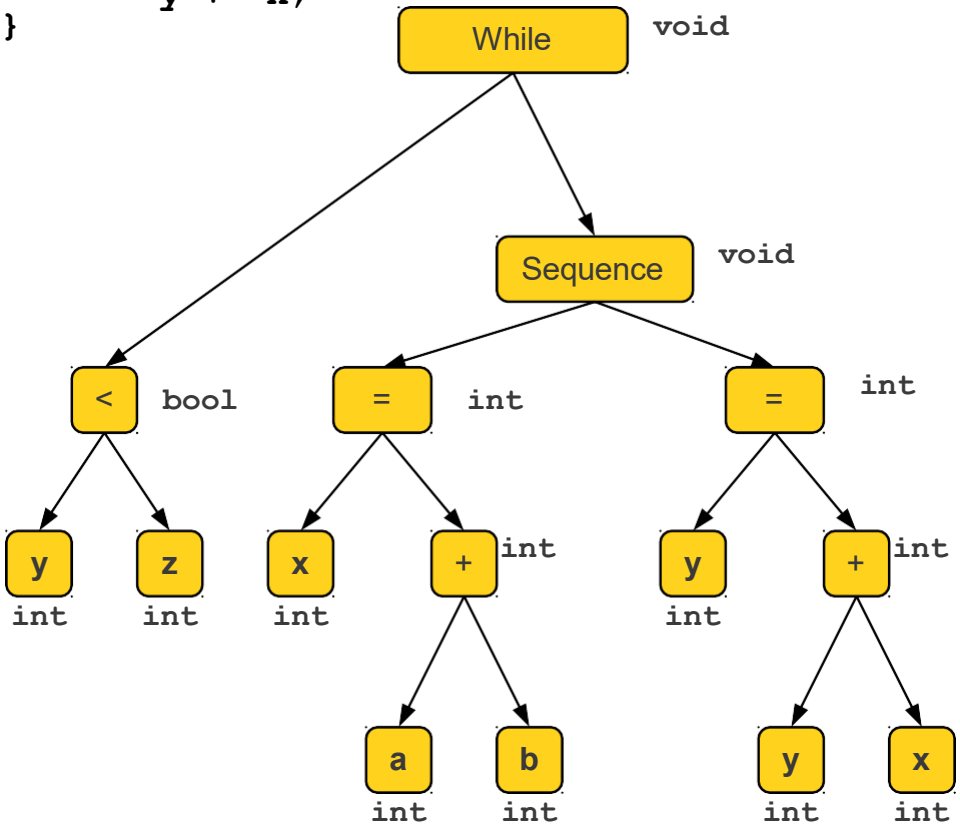


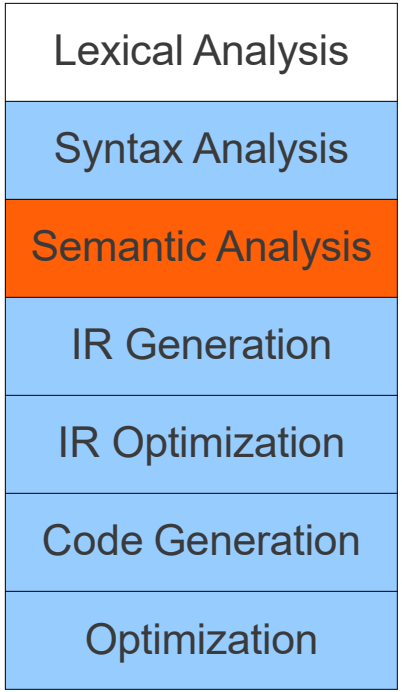
```
while (y < z) {
   int x = a + b;
      y += x;
         }
```



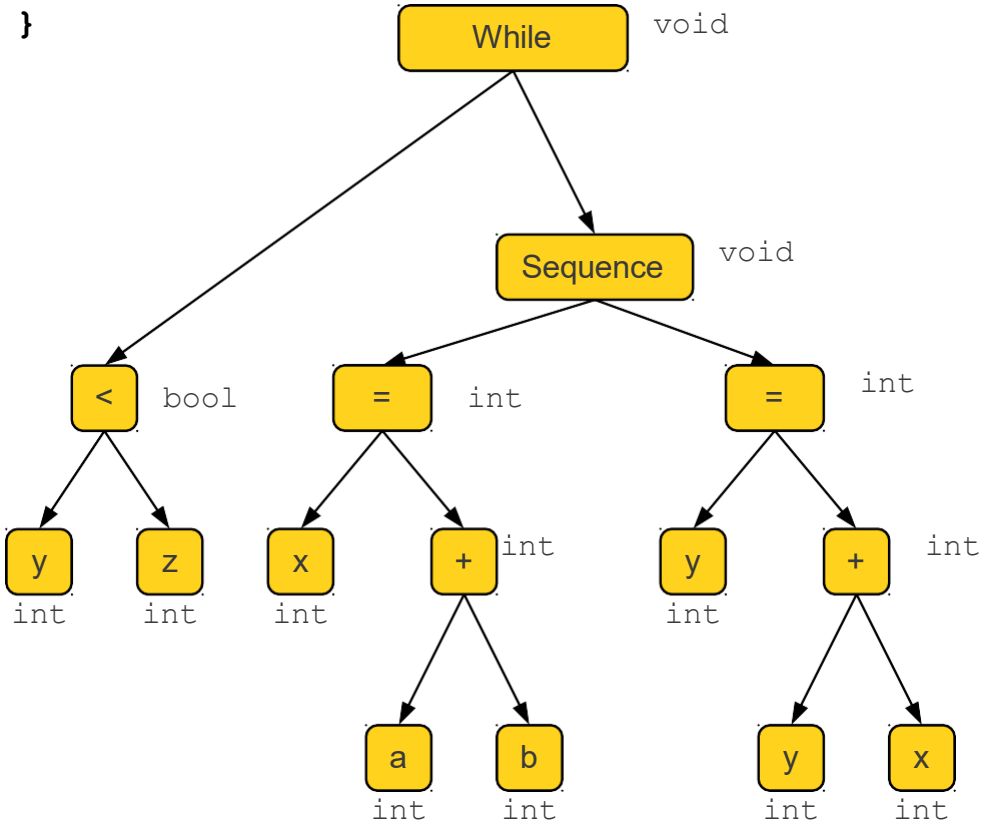


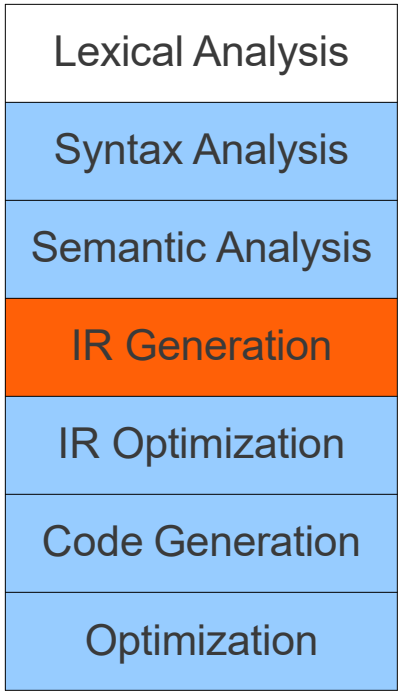
```
  while (y < z)
{  int x = a + b;
       y += x;
}
```





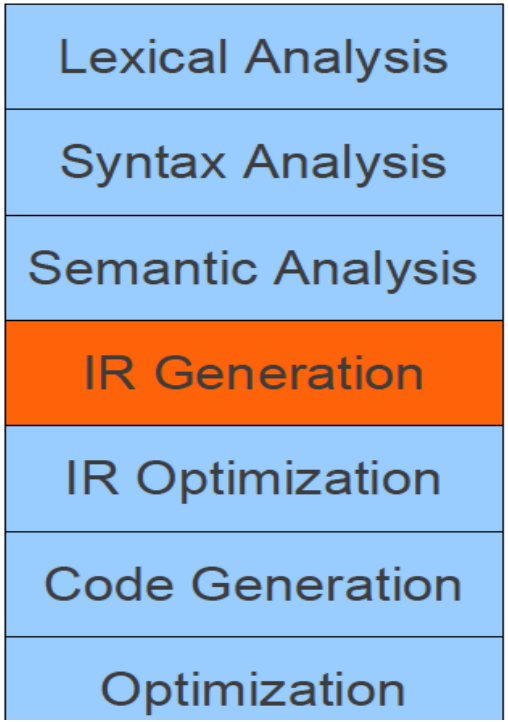
```
  while (y < z)
{  int x = a + b;
   y += x;
}
```





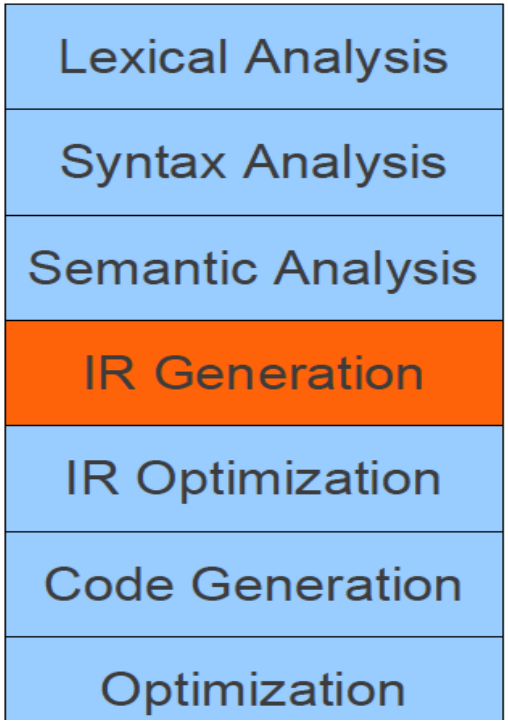
```
while (y < z) {
    int x = a + b;
    y += x;
}
```

```
Loop: x    = a + b
      y    = x + y
      _t1  = y < z
      if _t1 goto Loop
```



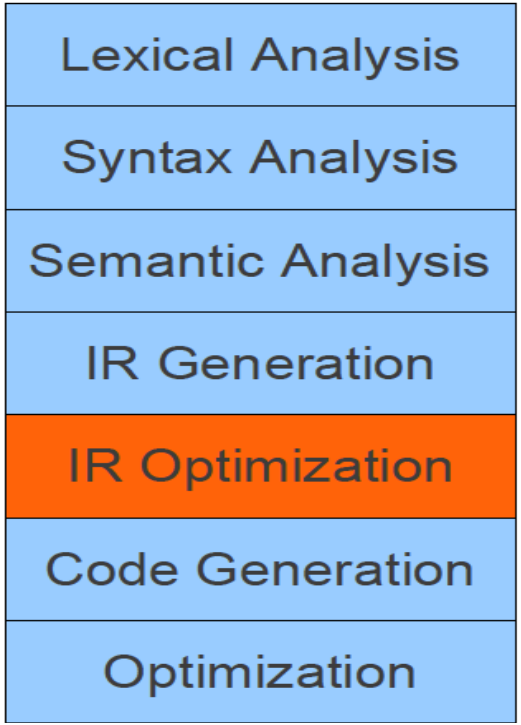
```
while (y < z) {
    int x = a + b;
    y += x;
}
```

```
Loop: x    = a + b
      y    = x + y
      _t1  = y < z
      if _t1 goto Loop
```



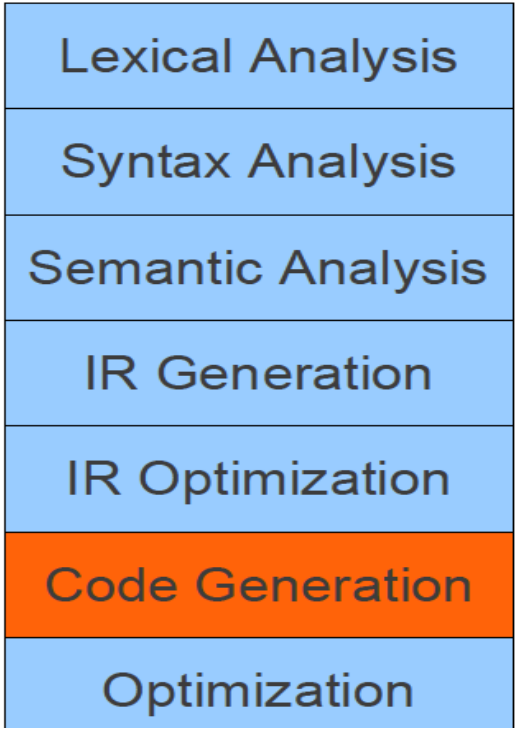
```
while (y < z) {
    int x = a + b;
    y += x;
}
```

```
        x    = a + b
Loop:   y    = x + y
        _t1  = y < z
        if _t1 goto Loop
```



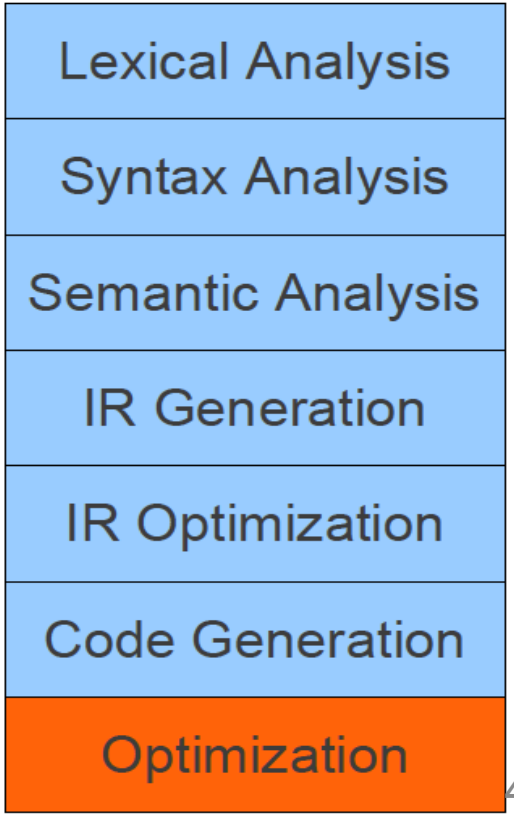
```
while (y < z) {
    int x = a + b;
    y += x;
}
```

```
        add $1, $2, $3
Loop:   add $4, $1, $4
        slt $6, $1, $5
        beq $6, loop
```



```
while (y < z) {
    int x = a + b;
    y += x;
}
```

```
       add $1, $2, $3
Loop:  add $4, $1, $4
       blt $1, $5, loop
```



LECTURE 3: FURTHER READING AND ASSIGNMENT

Kindly study the Chapter4 in the Textbook, **DES WATSON, on page 75-91**, for further background understanding on the lecture topic.